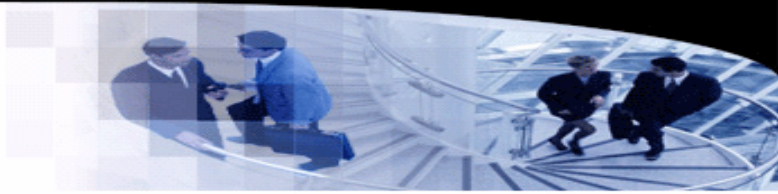
武漢大學
WUHAN UNIVERSITY

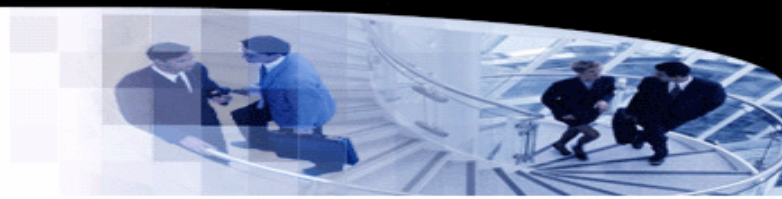


Programming Languages and Compilers

Lecture 11. Recursive Types

袁梦霆 (YUAN Mengting)
ymt@whu.edu.cn
School of Computer Science, Wuhan University

Recursion



- Syntax and semantics

→ `fix` *Extends* $\lambda_{\rightarrow}$ *(9-1)*

New syntactic forms

$t ::= \ldots$ *terms:*

$\texttt{fix}\ t$ *fixed point of* t

New evaluation rules $t \longrightarrow t'$

$$\texttt{fix}\ (\lambda x{:}T_1.t_2) \longrightarrow [x \mapsto (\texttt{fix}\ (\lambda x{:}T_1.t_2))]t_2 \quad \text{(E-FixBeta)}$$

$$\frac{t_1 \longrightarrow t_1'}{\texttt{fix}\ t_1 \longrightarrow \texttt{fix}\ t_1'} \quad \text{(E-Fix)}$$

New typing rules $\Gamma \vdash t : T$

$$\frac{\Gamma \vdash t_1 : T_1 \rightarrow T_1}{\Gamma \vdash \texttt{fix}\ t_1 : T_1} \quad \text{(T-Fix)}$$

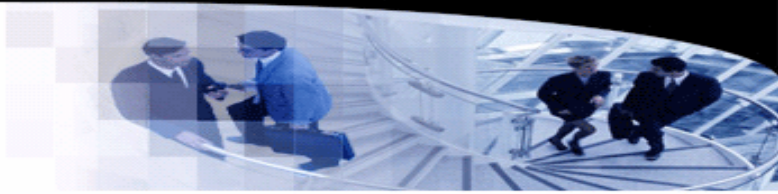
New derived forms

$$\texttt{letrec}\ x : T_1 = t_1\ \texttt{in}\ t_2 \stackrel{\text{def}}{=} \texttt{let}\ x = \texttt{fix}\ (\lambda x : T_1.t_1)\ \texttt{in}\ t_2$$

```
ff = λieio:{iseven:Nat→Bool, isodd:Nat→Bool}.
       {iseven = λx:Nat.
                   if iszero x then true
                   else ieio.isodd (pred x),
        isodd = λx:Nat.
                   if iszero x then false
                   else ieio.iseven (pred x)};
▸ ff : {iseven:Nat→Bool,isodd:Nat→Bool} →
       {iseven:Nat→Bool, isodd:Nat→Bool}

  r = fix ff;
▸ r : {iseven:Nat→Bool, isodd:Nat→Bool}

  iseven = r.iseven;
▸ iseven : Nat → Bool

  iseven 7;
▸ false : Bool
```

Lists

- Syntax and semantics



→ 𝔹 List — *Extends $\lambda_{\to}$ (9-1) with booleans (8-1)*

New syntactic forms

t	::=	...	*terms:*
		nil[T]	*empty list*
		cons[T] t t	*list constructor*
		isnil[T] t	*test for empty list*
		head[T] t	*head of a list*
		tail[T] t	*tail of a list*
v	::=	...	*values:*
		nil[T]	*empty list*
		cons[T] v v	*list constructor*
T	::=	...	*types:*
		List T	*type of lists*

New evaluation rules $\boxed{t \longrightarrow t'}$

$$\frac{t_1 \longrightarrow t_1'}{\texttt{cons[T]}\ t_1\ t_2 \longrightarrow \texttt{cons[T]}\ t_1'\ t_2} \quad \text{(E-Cons1)}$$

$$\frac{t_2 \longrightarrow t_2'}{\texttt{cons[T]}\ v_1\ t_2 \longrightarrow \texttt{cons[T]}\ v_1\ t_2'} \quad \text{(E-Cons2)}$$

$$\texttt{isnil[S] (nil[T])} \longrightarrow \texttt{true} \quad \text{(E-IsnilNil)}$$

$$\texttt{isnil[S] (cons[T]}\ v_1\ v_2\texttt{)} \longrightarrow \texttt{false} \quad \text{(E-IsnilCons)}$$

$$\frac{t_1 \longrightarrow t_1'}{\texttt{isnil[T]}\ t_1 \longrightarrow \texttt{isnil[T]}\ t_1'} \quad \text{(E-Isnil)}$$

$$\texttt{head[S] (cons[T]}\ v_1\ v_2\texttt{)} \longrightarrow v_1 \quad \text{(E-HeadCons)}$$

$$\frac{t_1 \longrightarrow t_1'}{\texttt{head[T]}\ t_1 \longrightarrow \texttt{head[T]}\ t_1'} \quad \text{(E-Head)}$$

$$\texttt{tail[S] (cons[T]}\ v_1\ v_2\texttt{)} \longrightarrow v_2 \quad \text{(E-TailCons)}$$

$$\frac{t_1 \longrightarrow t_1'}{\texttt{tail[T]}\ t_1 \longrightarrow \texttt{tail[T]}\ t_1'} \quad \text{(E-Tail)}$$

New typing rules $\boxed{\Gamma \vdash t : T}$

$$\Gamma \vdash \texttt{nil}\ [T_1] : \texttt{List}\ T_1 \quad \text{(T-Nil)}$$

$$\frac{\Gamma \vdash t_1 : T_1 \qquad \Gamma \vdash t_2 : \texttt{List}\ T_1}{\Gamma \vdash \texttt{cons[}T_1\texttt{]}\ t_1\ t_2 : \texttt{List}\ T_1} \quad \text{(T-Cons)}$$

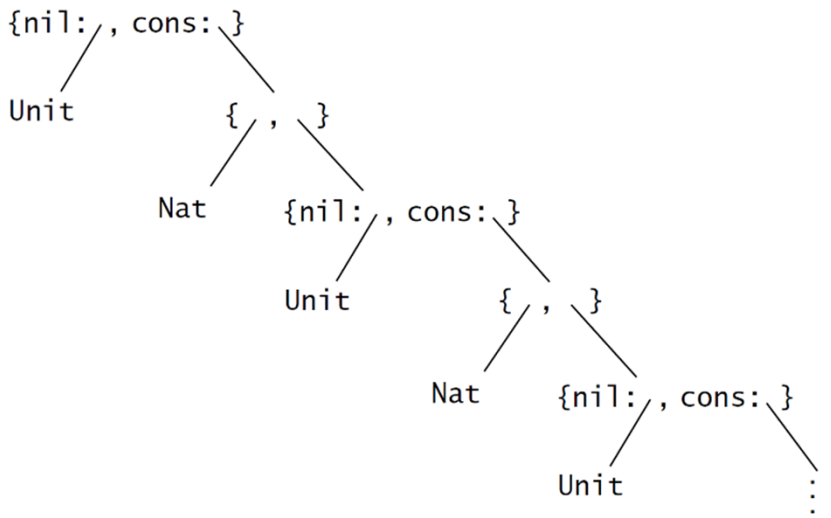
$$\frac{\Gamma \vdash t_1 : \texttt{List}\ T_{11}}{\Gamma \vdash \texttt{isnil[}T_{11}\texttt{]}\ t_1 : \texttt{Bool}} \quad \text{(T-Isnil)}$$

$$\frac{\Gamma \vdash t_1 : \texttt{List}\ T_{11}}{\Gamma \vdash \texttt{head[}T_{11}\texttt{]}\ t_1 : T_{11}} \quad \text{(T-Head)}$$

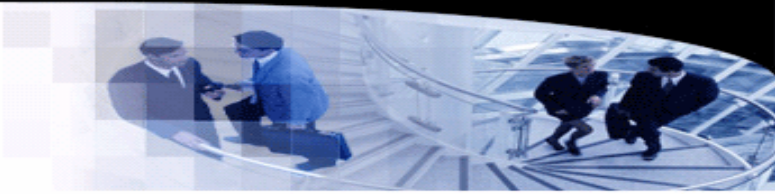
$$\frac{\Gamma \vdash t_1 : \texttt{List}\ T_{11}}{\Gamma \vdash \texttt{tail[}T_{11}\texttt{]}\ t_1 : \texttt{List}\ T_{11}} \quad \text{(T-Tail)}$$

Lists

- Types
  - We use `List T` to construct list types.
  - How to explain `List T` in mathematics (so the type checker can use the type information)3yhu?
  - If we unfold a list, the type of the list is unfolded as well
  - How to represent this infinite structure?
  - Example:
    - `NatList=<nil:Unit,cons:{…,…}>`
    - `NatList=<nil:Unit,cons:{Nat,…}>`
    - `NatList=<nil:Unit,cons:{Nat,NatList}>`

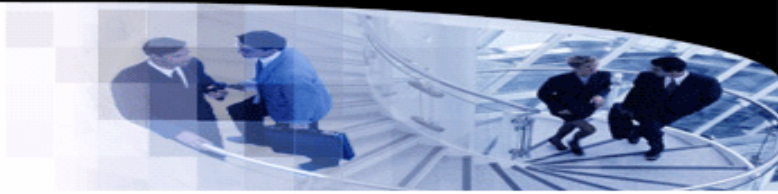


Lists



- Types
  - We introduce a recursion operator μ to represent recursive types.
    - `NatList = μX. <nil:Unit, cons:{Nat,X}>`
  - Let NatList be the infinite type satisfying the equation `NatList = μX. <nil:Unit, cons:{Nat,X}>`.
  - Two different formalizations: equi-recursive and iso-recursive.
  - Differing in the amount of help that the programmer is expected to give to the type checker in the form of type annotations.

Lists

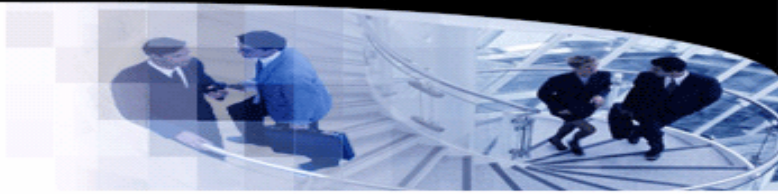


- Examples of lists
  - `nil`: the empty list.
  - `cons`: a constructor that constructs a new list by adding an element at the front of an existing list.
  - `isnil` checks whether the list is empty.
  - `hd` and `tl` retrieves the head element of the tail list of a given list.

```
nil = <nil=unit> as NatList;
▸ nil : NatList

cons = λn:Nat. λl:NatList. <cons={n,l}> as NatList;
▸ cons : Nat → NatList → NatList
```

```
isnil = λl:NatList. case l of
                      <nil=u> ⇒ true
                    | <cons=p> ⇒ false;
▸ isnil : NatList → Bool

hd = λl:NatList. case l of <nil=u> ⇒ 0 | <cons=p> ⇒ p.1;
▸ hd : NatList → Nat

tl = λl:NatList. case l of <nil=u> ⇒ l | <cons=p> ⇒ p.2;
▸ tl : NatList → NatList
```

Lists



- Examples of lists
  - sum calculates the summation of a given empty list.

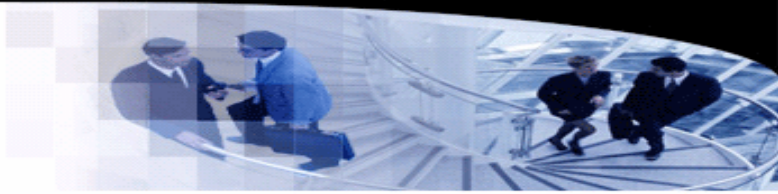
```
sumlist = fix (λs:NatList→Nat. λl:NatList.
                 if isnil l then 0 else plus (hd l) (s (tl l)));
```

```
▸ sumlist : NatList → Nat
```

```
mylist = cons 2 (cons 3 (cons 5 nil));
sumlist mylist;
```

```
▸ 10 : Nat
```

Examples of Recursive Types



- Hungry functions
  - A **hungry function** accepts an argument and returns a hungry function.
  - Hungry = μA. Nat→A

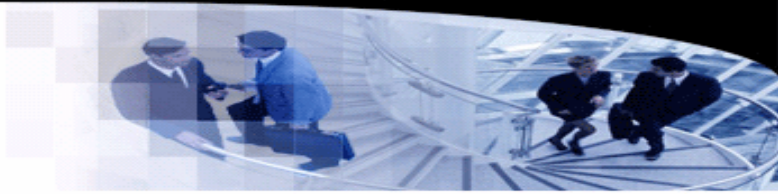
```
f = fix (λf: Nat→Hungry. λn:Nat. f);
```

▸ f : Hungry

```
f 0 1 2 3 4 5;
```

▸ <fun> : Hungry

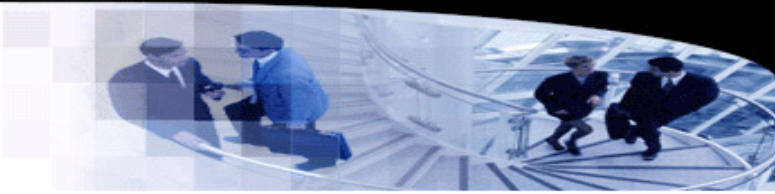
Examples of Recursive Types



- Streams
  - A stream function returns a natural number and a new stream function.
  - `Stream = μA. Unit→{Nat,A}`

```
hd = λs:Stream. (s unit).1;
▸ hd : Stream → Nat

tl = λs:Stream. (s unit).2;
▸ tl : Stream → Stream

upfrom0 = fix (λf: Nat→Stream. λn:Nat. λ_:Unit. {n,f (succ n)}) 0;
▸ upfrom0 : Stream

hd upfrom0;
▸ 0 : Nat

hd (tl (tl (tl upfrom0)));
▸ 3 : Nat
```

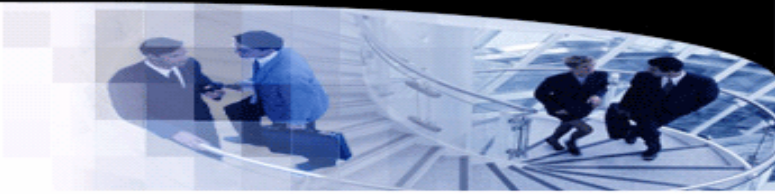
Examples of Recursive Types



- Streams
  - Streams can be used to represent processes.
  - `Process = μA. Nat→{Nat,A}`

```
p = fix (λf: Nat→Process. λacc:Nat. λn:Nat.
           let newacc = plus acc n in
           {newacc, f newacc}) 0;
▸ p : Process

curr = λs:Process. (s 0).1;
▸ curr : Process → Nat

send = λn:Nat. λs:Process. (s n).2;
▸ send : Nat → Process → Process
```

Examples of Recursive Types

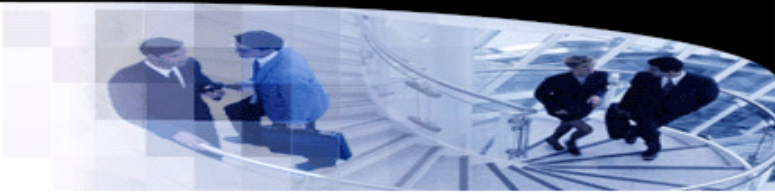


- Objects
  - For pure functional style objects, a method does some calculations and returns a new object.
  - Example:
    - `Counter = μC. {get:Nat, inc:Unit→C, dec:Unit→C}`

```
c = let create = fix (λf: {x:Nat}→Counter. λs: {x:Nat}.
                        {get = s.x,
                         inc = λ_:Unit. f {x=succ(s.x)},
                         dec = λ_:Unit. f {x=pred(s.x)} })
    in create {x=0};

▸ c : Counter
```

```
c1 = c.inc unit;
c2 = c1.inc unit;
c2.get;

▸ 2 : Nat
```

Formalization



- Fold and unfold
  - Two formalization strategies: the equi-recursive and iso-recursive approaches.
  - The essential difference between them is captured in their response to a simple question: What is the relation between the type $\mu X.T$ and its one-step unfolding?
  - equi-recursive:
    - These two type expressions as definitionally equal—interchangeable in all contexts—since they stand for the same infinite tree.
  - iso-recursive:
    - Takes a recursive type and its unfolding as different, but isomorphic.